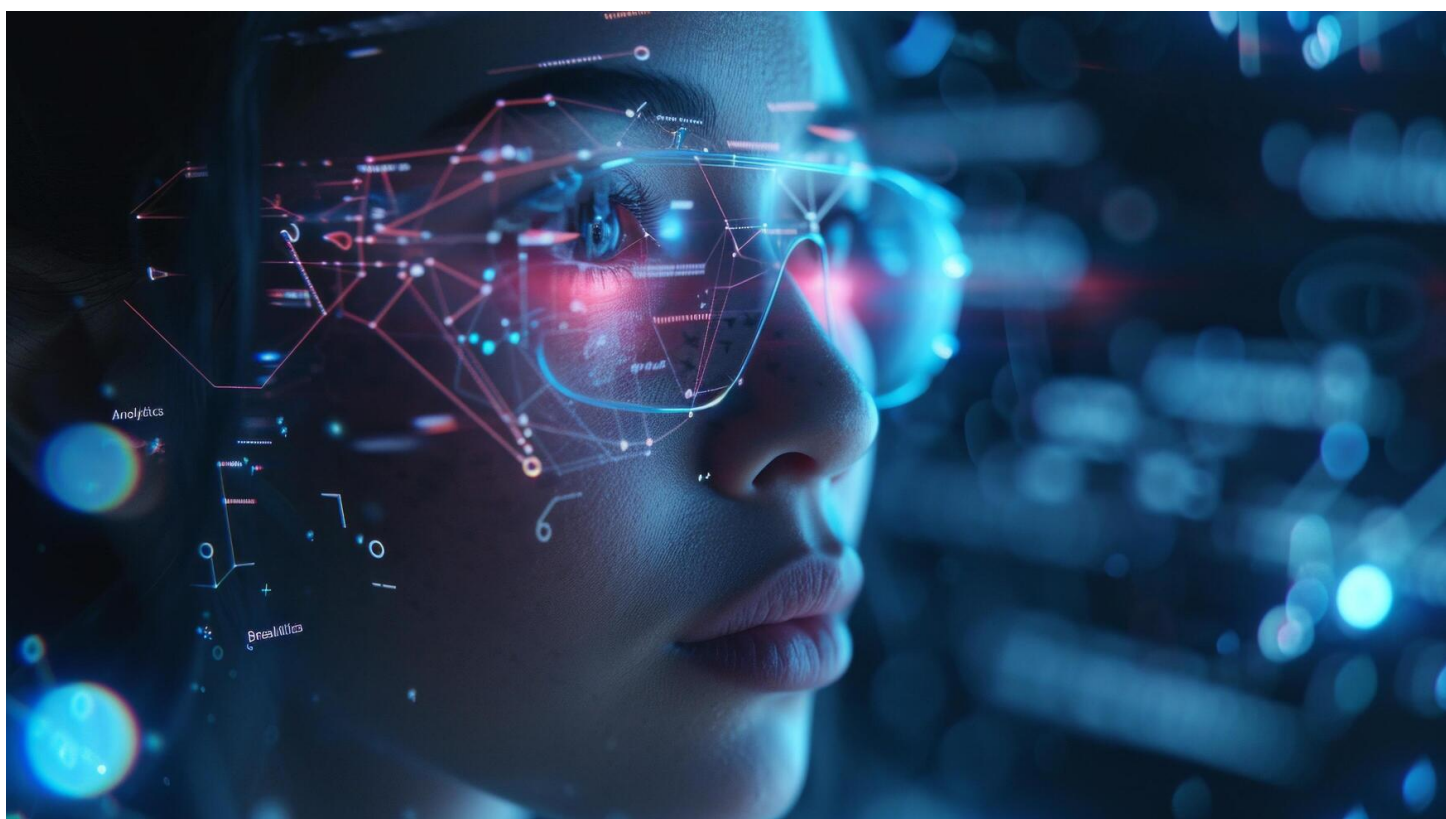


Sistema de Seguridad implementado en LabVIEW con comunicación de APIs

Daniel Alejandro Martínez Lara

May 31, 2026



MATERIA: LABORATORIO DE INSTRUMENTACIÓN VIRTUAL
DOCENTE: MEJÍA CARLOS MARCELA, L.S.C.

Contents

1	Introducción	2
2	Objetivos	3
3	Programa Principal	3
3.1	Programas de Python (Reconocimiento del Usuario)	4
3.2	Programas de Python (Reconocimiento de equipo)	5
3.3	Interfaz de LabVIEW (Conexión con Python)	6
3.4	Programa en LabVIEW pt.1	6
3.5	Programa en LabVIEW pt.2	8
3.6	Programa en LabVIEW pt.3	9
4	Funciones Adicionales	10
4.1	Agregar Usuario	10
4.2	Eliminar Usuario	11
5	Panel Frontal [Resultado]	11
6	Conclusiones	11

1 Introducción

Durante el desarrollo de la materia de LabVIEW se tocó el tema acerca de la implementación de APIs en el programa, facilitando procesos de reconocimiento de imágenes ya que primeramente empezamos con el sistema VISION integrado en LabVIEW, pero con la limitación de lo impreciso que podía ser el programa.

Con esta migración a un nuevo sistema de reconocimiento es que comenzó un interés por estos sistemas. Ya que esta aplicación trata del trabajo en la curiosidad de cómo podía implementarse a un sistema de seguridad.


©2021 @vintedigital

2 Objetivos

Para este proyecto tenemos los siguientes objetivos a cumplir durante el desarrollo del programa:

- Implementación de escaneo de código QR para el programa.
- Verificación de existencia del usuario dentro de una base de datos basada en Google Sheets.
- Verificación de equipo de seguridad del usuario para autorizar su acceso.
- Registro de cada intento de acceso y cada acceso autorizado por parte de los usuarios.

3 Programa Principal

Esta sección se separó en distintas etapas donde se explica en profundidad cada función del programa y del porqué se tomaron las decisiones de usar librerías y funciones dentro de LabVIEW y Python.

Principalmente se implementó el uso de Python para librerías como gspread para la conexión sencilla entre tokens y usuario ya que en LabVIEW se topaba con la dificultad de refrescar el token cada cierto tiempo, permitiendo así al usuario eliminar dicha preocupación y únicamente ocuparse del funcionamiento del programa.

3.1 Programas de Python (Reconocimiento del Usuario)

Como primera implementación se utilizó el siguiente código de Python.

```
import gspread
from datetime import datetime
import random
import pyttsx3

voz = pyttsx3.init()
voz.setProperty('rate', 150) # Velocidad de la voz
voz.setProperty('volume', 1) # Volumen (0.0 a 1.0)
voz.setProperty('voice', 'es-ES') # Cambia a voz en español (puede variar según el sistema operativo)

def verificar_y_registrar(id_escaneado):
    # Esto busca automáticamente 'credentials.json' en tu carpeta
    # y genera un 'token.json' para no pedirte permiso a cada rato
    cuenta = gspread.oauth(
        credentials_filename='C:/Users/Gamer/OneDrive/Documentos/Universidad/8vo_Semestre/Lab_View-2/Proyecto-Final/codigos-py/credentials.json',
        authorized_user_filename='C:/Users/Gamer/OneDrive/Documentos/Universidad/8vo_Semestre/Lab_View-2/Proyecto-Final/codigos-py/token.json'
    )

    # Abre tu Excel (Asegúrate de que el nombre sea EXACTAMENTE igual)
    bitacora = cuenta.open("Laboratorio-login")
    hoja_usuarios = bitacora.worksheet("Usuarios")
    hoja_log = bitacora.worksheet("Login")

    # 1. Buscamos al usuario en la pestaña de autorizados
    celda = hoja_usuarios.find(str(id_escaneado))

    # Si la celda es 'None', significa que el ID no está en el Excel
    if celda is None:
        ahora = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        hoja_log.append_row([ahora, id_escaneado, "Desconocido", "DENEGADO", "ID no registrado"])
        voz.say("Acceso denegado, ID no registrado")
        voz.runAndWait()
        return False

    # Si el código llega hasta aquí, es porque sí encontró la celda
    fila = celda.row
    datos = hoja_usuarios.row_values(fila)

    nombre = datos[1]
    estatus = datos[2]

    # 2. Registramos el intento en la bitácora
    ahora = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    if estatus.lower() == "activo":
        hoja_log.append_row([ahora, id_escaneado, nombre, "PERMITIDO", "Filtro 1 superado"])
        voz.say("Acceso autorizado, bienvenido {nombre}")
        voz.runAndWait()
        return True
    else:
        hoja_log.append_row([ahora, id_escaneado, nombre, "DENEGADO", "Usuario inactivo"])
        voz.say("Acceso denegado, usuario inactivo")
        voz.runAndWait()
        return False
```

Figure 1: Código de acceso al usuario.

Este código se encarga de recibir como entrada un valor numérico, para después acceder a las credenciales del usuario para poder inicializar los servicios de Google.

Una vez con esto entra dentro de Google Spreadsheets para acceder a la base de datos del sistema, guarda una variable para acceder a la bitácora de accesos y otra variable para la hoja de usuarios dentro del sistema.

En este caso estamos buscando a un usuario por lo que leemos la hoja para encontrar el id escaneado, en caso de que no lo encuentre nos manda un mensaje de error diciéndonos que el usuario no se encuentra dentro de la base de datos de usuarios registrados.

En caso de encontrar al usuario guardamos el número de fila y columna y extraemos los datos de esa fila para darle la bienvenida al usuario, además de acceder a la hoja encargada de la bitácora para poder guardar el acceso al PRIMER filtro.

3.2 Programas de Python (Reconocimiento de equipo)

Como segunda implementación se utilizó el siguiente código.

```

11 def detect_ppe(ruta_imagen,id_usuario):
12     ..
13     with open(ruta_imagen, 'rb') as imagen:
14         respuesta = cliente.detect_protective_equipment(
15             Image={'Bytes': imagen.read()},
16             # Corregimos para pedir SOLAMENTE lo que Amazon sabe leer
17             SummarizationAttributes={
18                 'MinConfidence': 80,
19                 'RequiredEquipmentTypes': ['FACE_COVER', 'HAND_COVER']
20             }
21         )
22     ..
23     resumen = respuesta.get('Summary', {})
24     ids_sin_equipo = resumen.get('PersonsWithoutRequiredEquipment', [])
25     ..
26     if not ids_sin_equipo:
27         voz.say("Acceso autorizado, equipo completo")
28         voz.runAndWait()
29         hoja_log.append_row([ahora, id_usuario, nombre, "PERMITIDO", "Filtro 2 superado"])
30         return True
31     ..
32     faltantes = []
33     personas_detalladas = respuesta.get('Persons', [])
34     ..
35     for persona in personas_detalladas:
36         if persona.get('Id') in ids_sin_equipo:
37             partes_cuerpo = persona.get('BodyParts', [])
38             for parte in partes_cuerpo:
39                 equipo = parte.get('EquipmentDetections', [])
40                 ..
41                 if not equipo:
42                     nombre_parte = parte.get('Name')
43                     # Va solo evaluamos Cara y Manos
44                     if nombre_parte == 'FACE':
45                         faltantes.append("Lentes de seguridad")
46                     elif nombre_parte in ['LEFT_HAND', 'RIGHT_HAND']:
47                         faltantes.append("Guantes de nitrilo")
48             ..
49     respuesta_labels= cliente.detect_labels(
50         Image={'Bytes': open(ruta_imagen, 'rb').read()},
51         MaxLabels=20,
52         MinConfidence=75
53     )
54     ..
55     # Extraemos solo los nombres de las cosas que detectó
56     etiquetas_encontradas = [etiqueta['Name'] for etiqueta in respuesta_labels['Labels']]
57     ..
58     # Buscamos variaciones de bata en inglés
59     tiene_bata = ('Lab Coat' in etiquetas_encontradas or
60                 'Coat' in etiquetas_encontradas or
61                 'Medical' in etiquetas_encontradas)
62     ..
63     if not tiene_bata:
64         faltantes.append("Bata de laboratorio")
65     ..
66     faltantes_unicos = list(set(faltantes))
67     voz.say("Acceso denegado, favor de portar " + ", ".join(faltantes_unicos))
68     voz.runAndWait()
69     texto_alarma = "Equipo incompleto, favor de portar " + ", ".join(faltantes_unicos)
70     hoja_log.append_row([ahora, id_usuario, nombre, "DENEGADO", "Filtro 2 NO superado"])
71     return False
72     ..

```

Figure 2: Código de Verificación de equipo.

Para este caso el código se encarga de recibir la imagen a la cual le va a aplicar reconocimiento, el `id_usuario` se recibe para un sistema de voz implementado por medio de la librería **pyttsx3**. Con la imagen obtenida el programa la abre y define los atributos con los cuales va a ser el score mínimo para detectar el objeto, en este caso checaremos que el usuario lleve cubierta la cara con lentes de seguridad y guantes para laboratorio.

Guardamos los resultados del escaneo que realiza la API y en caso de que la variable "ids_sin_equipo" esté vacía significa que el usuario cuenta con los equipos necesarios para entrar al laboratorio y realizamos un escaneo adicional para verificar que la persona cuente con una bata de seguridad. Cuando se cuenta con el equipo completo el sistema nos dará acceso, en caso contrario el sistema nos comentará del equipo que hace falta portar.

3.3 Interfaz de LabVIEW (Conexión con Python)

La conexión con LabVIEW se realizó de la siguiente forma. La función conocida como "Python Node"

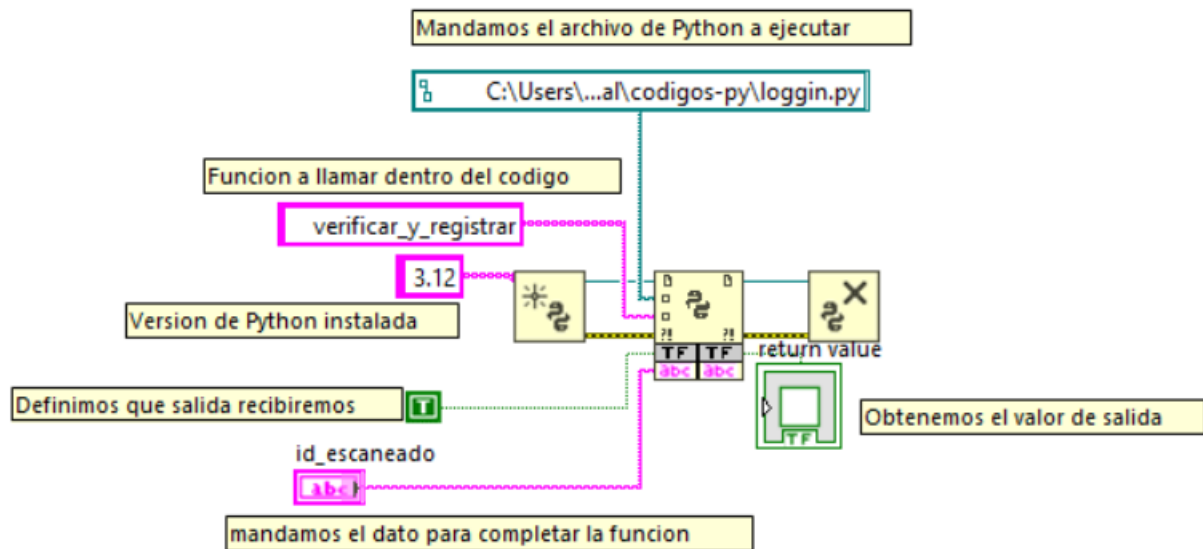


Figure 3: Función de Python Node.

es una función que permite la interacción entre LabVIEW y Python en la cual inicializamos la sesión de Python donde definimos la versión de Python que utilizaremos para después mandar a llamar el archivo que ejecutaremos, a su vez le decimos qué función de nuestro programa desea que utilicemos y finalmente le definimos el tipo de salida que nos regresará el programa y el indicador de entrada que recibirá el código.

3.4 Programa en LabVIEW pt.1

El programa de LabVIEW nos recibe con la siguiente comparación de booleanos para verificar a qué estado nos mandará, ya que este programa está regido por una máquina de estados.

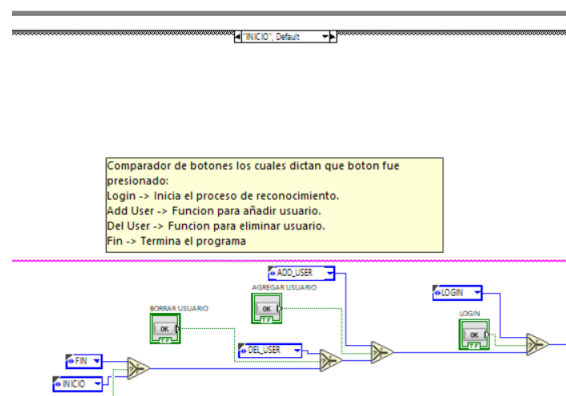


Figure 4: Estado de "INICIO" del programa.

Cuando escogemos el botón de "Search" iniciamos el proceso de login, donde el programa realiza la siguiente función.

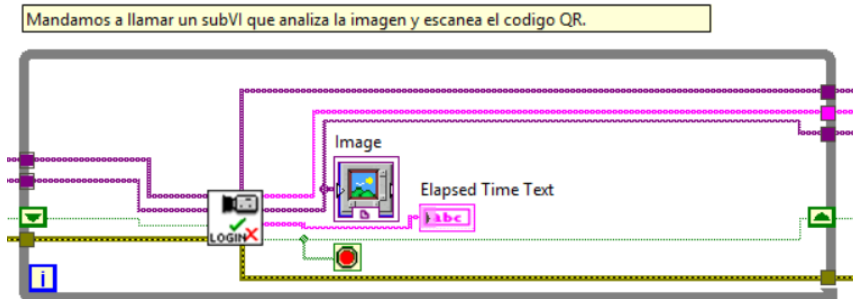


Figure 5: Utilización de subVI especializado.

El programa recibe la imagen actual del programa para entrar a un subVI que se encarga de analizar el código QR que el usuario tiene.

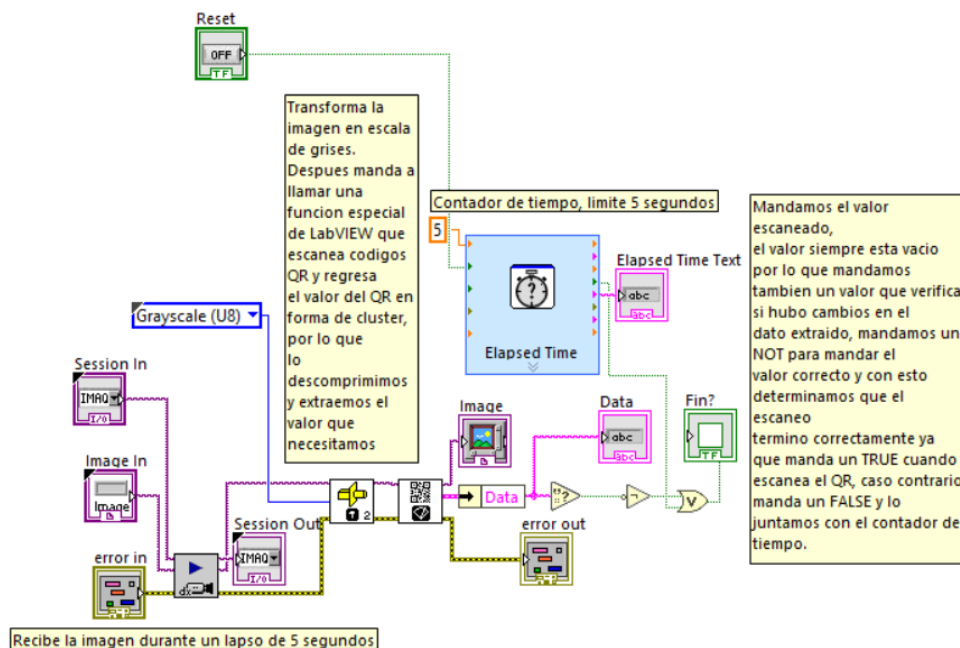


Figure 6: Escaneo de QR.

Este subVI recibe la imagen continuamente durante 5 segundos como límite, en caso de superar esos 5 segundos el escaneo termina y regresa un valor vacío. En caso de que encuentre un código QR capaz de escanear lo escanea y regresa en forma de cluster el valor que encuentra dentro del QR, por lo que descomprimos lo que regresa la función integrada a LabVIEW y sacamos únicamente el valor escaneado, cuando este valor cambia está conectado a una función que está continuamente checando cuando cambia el valor de "Data" en caso de cambiar manda un booleano y lo invertimos con un NOT para evitar errores.

3.5 Programa en LabVIEW pt.2

Una vez que se obtiene algún ID o incluso puede mandar un número vacío (El cual Python tomará como 0), se manda a llamar el programa que realiza un barrido para buscar la ID del usuario y manda una respuesta.

En caso de encontrar un usuario nos dará acceso a la segunda fase para realizar login, caso contrario nos regresará al inicio y nos dirá que el usuario no se encontró, TODO movimiento que hagamos se guardará en la bitácora de accesos ya sean accesos autorizados o denegados.

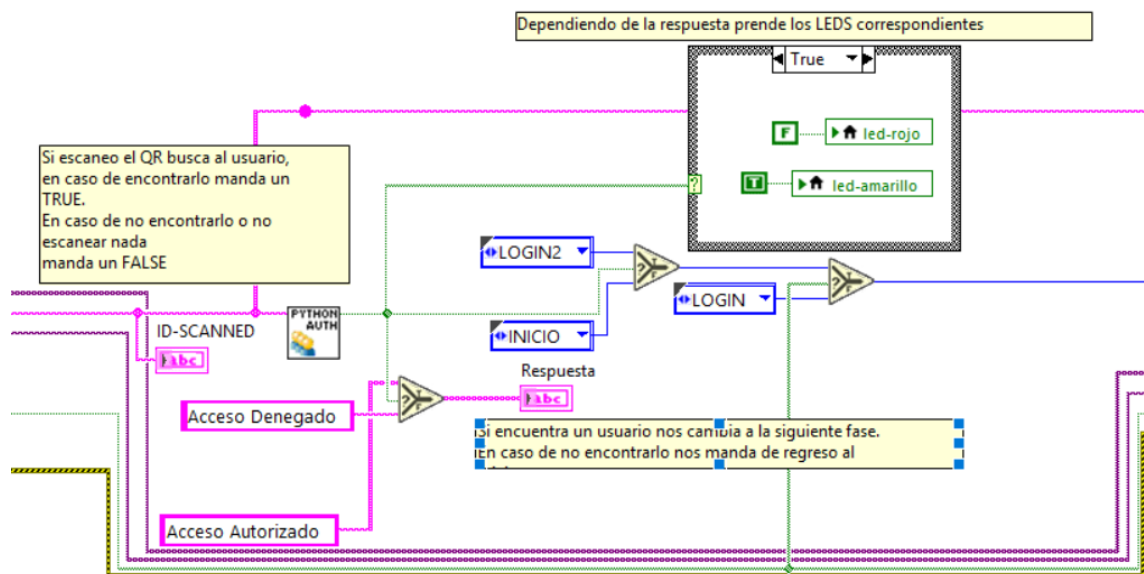


Figure 7: Escaneo de ID.

3.6 Programa en LabVIEW pt.3

Cuando el usuario puede entrar a la fase 2 del sistema de logueo, se le da indicaciones que se quede de forma visible frente a la cámara ya que se le tomará una foto de ese momento preciso en el que acabe el mensaje dado por un script de Python.

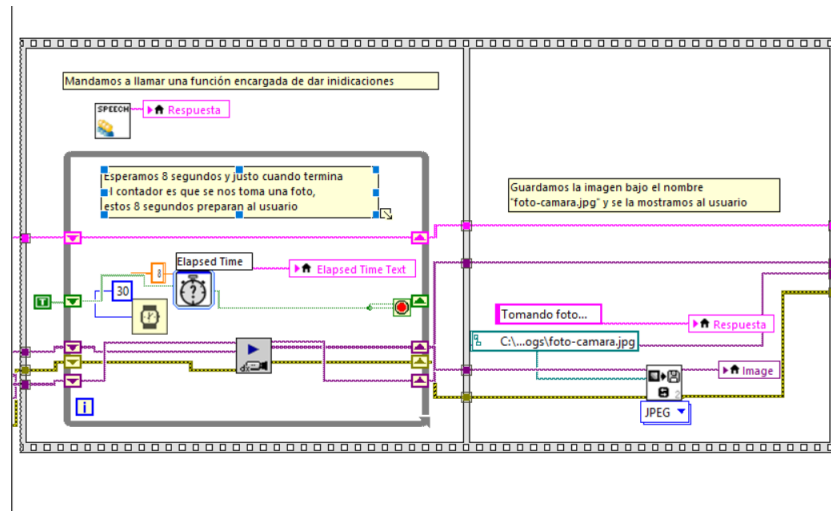


Figure 8: Login fase 2 pt1.

El sistema entra en un conteo de 8 segundos en los cuales la persona se tiene que preparar, una vez pasado ese tiempo se le tomará una foto. Dicha foto se guardará en forma de archivo con nombre "foto-camara.jpg" la cual se le mostrará al usuario.

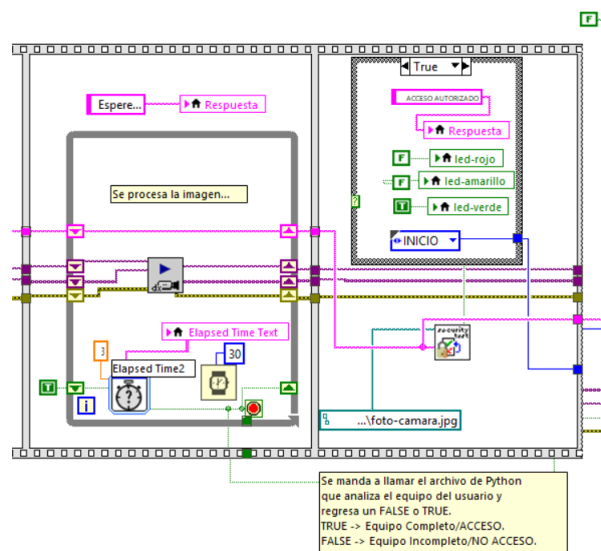


Figure 9: Login fase 2 pt2.

La foto será procesada, además de esperar 3 segundos para no saturar las tareas del sistema, después de ese tiempo se manda a llamar el script que explicamos anteriormente el cual analizará

la imagen y nos entregará si el usuario cuenta con el equipo completo o hace falta que porte un equipamiento extra.

4 Funciones Adicionales

Como funciones adicionales se agregaron 2 para gestionar a los usuarios dentro del laboratorio dando esa libertad de eliminar o agregar usuarios a la base de datos encargada de monitorear a los usuarios que pueden entrar al laboratorio.

4.1 Agregar Usuario

La siguiente función es llamada por medio de Python ya que nuevamente es más fácil utilizar los servicios de Google, por lo que explicaremos el script.

```
def agregar_usuario(nombre,status):
    cuenta = gspread.oauth(
        credentials_filename="C:/Users/Gamer/OneDrive/Documentos/Universidad/8vo_Semestre/Lab_View-2/Proyecto-Final/codigos-py/credentials.json",
        authorized_user_filename="C:/Users/Gamer/OneDrive/Documentos/Universidad/8vo_Semestre/Lab_View-2/Proyecto-Final/codigos-py/token.json"
    )

    bitacora = cuenta.open("Laboratorio-login")
    hoja_usuarios = bitacora.worksheet("Usuarios")
    id_registrados = hoja_usuarios.col_values(1)[1:]
    id_nuevo=""
    while True:
        id_temp= sum(ord(letra) for letra in nombre.replace(" ", ""))
        rand = random.randint(1,400)
        id_nuevo = str(id_temp) + str(rand)
        if id_nuevo not in id_registrados:
            break
        print(id_registrados)

    nombre = nombre.upper()
    status = status.upper()

    hoja_usuarios.append_row([id_nuevo, nombre, status])
    voz.say(f"Usuario agregado: {nombre}")
    voz.runAndWait()

    return f"USUARIO AGREGADO|{nombre}|ID: {id_nuevo}"
```

Figure 10: Login fase 2 pt2.

Este script nuevamente accede a la base de datos pero dentro de su función recibe el nombre del usuario y manda automáticamente un estatus "activo", con el nombre de la persona y se va a la fila que esté vacía para poder agregar los datos de la persona, además de generar un ID basado en código ASCII de cada letra del nombre del usuario y después agregarle un número aleatorio, después verifica que la ID no esté registrada en el sistema, en caso de ya estar registrada sumamos un valor aleatorio hasta que encontremos un ID no utilizado y lo registra en la base de datos.

4.2 Eliminar Usuario

La siguiente función es llamada por medio de Python ya que nuevamente es más fácil utilizar los servicios de Google, por lo que explicaremos el script.

```
def eliminar_usuario(id_usuario):
    cuenta = gspread.oauth(
        credentials_filename='C:/Users/Gamer/OneDrive/Documentos/Universidad/8vo_Semestre/Lab_View-2/Proyecto-Final/codigos-py/credentials.json',
        authorized_user_filename='C:/Users/Gamer/OneDrive/Documentos/Universidad/8vo_Semestre/Lab_View-2/Proyecto-Final/codigos-py/token.json'
    )

    bitacora = cuenta.open("Laboratorio-login")
    hoja_usuarios = bitacora.worksheet("Usuarios")
    id_registrados = hoja_usuarios.col_values(1)[1:]

    if id_usuario not in id_registrados:
        voz.say("Error, ID no encontrado")
        voz.runAndWait()
        return "ERROR|ID no encontrado"
    else:
        fila = id_registrados.index(id_usuario) + 2
        nombre = hoja_usuarios.cell(fila, 2).value
        hoja_usuarios.delete_rows(fila)
        voz.say(f"Usuario eliminado: {nombre}")
        voz.runAndWait()

    return f"USUARIO ELIMINADO|ID: {id_usuario}"
```

Figure 11: Login fase 2 pt2.

Este script recibe únicamente la ID del usuario que se desea eliminar, posteriormente ingresa a la base de datos que mantiene a los usuarios activos y realiza un barrido el cual busca la ID del usuario, en caso de encontrarlo simplemente lo elimina, pero caso contrario nos menciona que la ID no se encontró.

5 Panel Frontal [Resultado]

Como resultado, se acomodó de la siguiente forma el panel frontal para permitir una interacción cómoda con el programa.



Figure 12: Panel Frontal.

6 Conclusiones

La elaboración de este proyecto permitió al usuario entender la interacción e implementación de APIs en Python y posteriormente su conexión con LabVIEW para así crear un programa interactivo que cumpla los objetivos esperados y propuestos para este proyecto.